

PYTHON — STRINGS

Understanding Text, Indexing, Slicing & String Operations

LEARN • BUILD • EXPLORE

01 Introduction to Strings

In Python, textual data is represented using **Strings**. A string is an ordered **sequence of characters** enclosed within quotation marks. Strings can contain letters, numbers, symbols, spaces, and even emojis!

💡 Why are Strings essential?

Programs constantly process text—reading user names, displaying messages, formatting reports, parsing web pages, or writing data to files. Strings are Python's primary tool for handling text.

02 Creating Strings

Python provides great flexibility by allowing you to create strings using single, double, or triple quotes.

Quote Type	Syntax Example	Primary Use Case
Single Quotes	<code>'Hello'</code>	Short strings, simple text tokens.
Double Quotes	<code>"Python Code"</code>	Standard text, or text containing single quotes (<code>"It's fine"</code>).
Triple Quotes	<code>'''Line 1\nLine 2'''</code> or <code>"""..."""</code>	Multi-line strings, docstrings, or text with mixed quotes.

03 String Variables

When you assign a string to a variable, Python creates a string object in memory and references it via that variable name.

```
# Storing string values in variables
name = "Nil"
language = "Python"
tagline = """LEARN
BUILD
EXPLORE"""
```

```
print(name)
print(language)
```

TERMINAL OUTPUT

```
Nil
Python
```

04 Accessing Characters (Indexing)

Because strings are ordered sequences, every character has a specific numerical address called an **index**. Python supports both **Positive Indexing** (left-to-right starting at 0) and **Negative Indexing** (right-to-left starting at -1).

05 String Slicing

Slicing extracts a substring by specifying a range of indices. The syntax is `string[start : end : step]`.

⚠ The Golden Rule of Slicing

The `end` index is **EXCLUDED**! Slicing collects characters from `start` up to `end - 1`.

```
text = "Python"

# 1. Standard Slice [0:3] -> Indices 0, 1, 2
print(text[0:3]) # Output: 'Pyt'

# 2. Slice from middle [2:6] -> Indices 2, 3, 4, 5
print(text[2:6]) # Output: 'thon'

# 3. Omit Start [:4] -> Starts from beginning (index 0)
print(text[:4]) # Output: 'Pyth'

# 4. Omit End [2:] -> Goes all the way to the end
print(text[2:]) # Output: 'thon'

# 5. Reverse String[::-1] -> Negative step reverses order
print(text[::-1]) # Output: 'nohtyP'

-----
TERMINAL OUTPUT
Pyt
thon
Pyth
thon
nohtyP
```

06 Concatenation

Joining two or more strings using the `+` operator.

```
first = "Nil"
last = "Labs"
name = first + " " + last
print(name)

-----
OUTPUT
Nil Labs
```

07 Repetition

Repeating a string multiple times using the `*` operator.

```
word = "Python "
result = word * 3
print(result)

-----
OUTPUT
Python Python Python
```

08 Essential String Methods

Methods are built-in functions executed on strings using **dot notation** (`string.method()`).

Method	Description	Code Example	Output
<code>.upper()</code>	Converts all characters to uppercase	<code>"nil".upper()</code>	"NIL"
<code>.lower()</code>	Converts all characters to lowercase	<code>"NIL".lower()</code>	"nil"
<code>.capitalize()</code>	Capitalizes 1st character only	<code>"python notes".capitalize()</code>	"Python notes"
<code>.title()</code>	Capitalizes 1st letter of every word	<code>"nil labs notes".title()</code>	"Nil Labs Notes"

09 The len() Function

The built-in `len()` function calculates the total number of characters in a string, including spaces, numbers, and punctuation marks.

```
text1 = "Python"
text2 = "NIL LABS" # Includes 1 space
```

```
print(len(text1)) # Output: 6
print(len(text2)) # Output: 8
```

TERMINAL OUTPUT

```
6
8
```

10 Membership Operators (in / not in)

You can easily check if a substring exists inside another string using the membership keywords `in` and `not in`. These return Booleans (`True` / `False`).

```
text = "Python Programming"
```

```
print("Py" in text) # Returns True
print("Java" in text) # Returns False
print("C++" not in text) # Returns True
```

TERMINAL OUTPUT

```
True
False
True
```

11 Escape Characters

An escape character starts with a backslash (`\`) and lets you insert special characters into strings that would otherwise cause syntax errors.

Escape Seq	Meaning	Code Example	Rendered Result
<code>\n</code>	New Line	<code>"Hello\nWorld"</code>	Hello World
<code>\t</code>	Horizontal Tab	<code>"A\tB"</code>	A B
<code>\"</code>	Double Quote	<code>"She said \"Hi\""</code>	She said "Hi"
<code>\'</code>	Single Quote	<code>'It\'s Python'</code>	It's Python

12 String Formatting (f-strings)

Introduced in Python 3.6, **f-strings** (Formatted String Literals) provide a clean, readable way to embed variables and expressions directly inside text. Prefix the string with `f` and put variables inside `{curly braces}`.

```
name = "Nil"
age = 17

# Clean f-string formatting
```

14 Common Beginner Mistakes

✗ MISTAKE 1: ATTEMPTING IN-PLACE MODIFICATION

```
text = "Python"
text[0] = "J"
# TypeError: 'str' object does not support item
assignment
```

✓ CORRECT: CREATE A NEW STRING

```
text = "Python"
text = "J" + text[1:]
print(text) # 'Jython'
```

✗ MISTAKE 2: MISSING STRING QUOTES

```
msg = Hello World
# SyntaxError: invalid syntax
```

✓ CORRECT: WRAP IN QUOTES

```
msg = "Hello World"
print(msg)
```

✗ MISTAKE 3: INDEX OUT OF RANGE

```
text = "Nil"
print(text[3])
# IndexError: string index out of range
```

✓ CORRECT: MAX INDEX IS LEN - 1

```
text = "Nil"
print(text[2]) # Output: 'l'
```

15 Practice Exercises

Solve these exercises on paper or in your Python IDLE before moving forward!

Exercise 1: Character Extraction

Indexing

Given `course = "Computer Science"`, write code to extract the first character, the last character, and the space character using negative indexing.

[Write your code solution here]

Exercise 2: Substring Slicing

Slicing

Given `phrase = "NIL LABS PYTHON"`, slice out only the word "LABS" and print it in reversed order.

[Write your code solution here]

Exercise 3: String Clean Up

Methods

Given `raw_data = " #python3_notes# "`, write expression to strip whitespace and replace "#" with an empty string.

[Write your code solution here]

Exercise 4: Membership & Length Check

Logic

Write an `if` statement checking if "LABS" is inside `brand = "NIL LABS"` AND its length is greater than 5.

[Write your code solution here]

16 Mini Challenge: Build a Simple Name Formatter

🎯 Challenge Task Specification

Write a Python script that takes a poorly formatted input string like `user_input = " nIL lAbS "` and performs the following operations:

1. Removes all extra leading and trailing whitespace.
2. Converts the name into proper Title Case ("Nil Labs").
3. Calculates and prints the cleaned string's total length.
4. Displays its first character and last character using indexing.
5. Outputs a final formatted greeting using an f-string!

[Attempt Your Mini Challenge Code Solution Below]

Hint: Combine `.strip()`, `.title()`, `len()`, `[0]`, `[-1]`, and `f"..."`

17 Quick Recap

⚡ Core Takeaways

- Strings are **immutable** character sequences.
- Positive indexing: `0` to `N-1`.
- Negative indexing: `-1` to `-N`.
- Slicing `[start:end]` excludes `end`.
- Methods like `.upper()` return new strings.

NIL LABS

LEARN • BUILD • EXPLORE

"Small steps. Big goals."